

# A Practical Guide to Adam optimizer

TOPIC -- ADAM OPTIMIZER

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

-- 01 --

$$[w_1 \ w_2] \leftarrow [w_1 \ w_2] - \eta \left[ \frac{\partial}{\partial w_1} (w_1 + w_2 x_i - y_i)^2 \ \frac{\partial}{\partial w_2} (w_1 + w_2 x_i - y_i)^2 \right] = [w_1 \ w_2] - \eta \left[ 2(w_1 + w_2 x_i - y_i) x_i \ (w_1 + w_2 x_i - y_i) \right].$$

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \leftarrow \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial}{\partial w_1} (w_1 + w_2 x_i - y_i)^2 \\ \frac{\partial}{\partial w_2} (w_1 + w_2 x_i - y_i)^2 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} 2(w_1 + w_2 x_i - y_i) x_i \\ 2(w_1 + w_2 x_i - y_i) \end{bmatrix}.$$

Note that in each iteration or update step, the gradient is only evaluated at a single  $x_i$ . This is the key difference between stochastic gradient descent and batched gradient descent. In general, given a linear regression  $\hat{y} = \sum_{k=1}^m w_k x_k$  problem, stochastic gradient descent behaves differently when  $m < n$  (underparameterized) and  $m \geq n$  (overparameterized). In the overparameterized case, stochastic gradient descent converges to  $\arg \min_{w: w^T x_k = y_k \ \forall k \in 1:n} \|w - w_0\|$ . That is, SGD converges to the interpolation solution with minimum distance from the starting  $w_0$ . This is true even when the learning rate remains constant. In the underparameterized case, SGD does not converge if learning rate remains constant.

-- 02 --

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients  $Q_i$  are sufficiently smooth. Let  $T > 0$  and  $g: \mathbb{R}^d \rightarrow \mathbb{R}$  be a sufficiently smooth test function. Then, there exists a constant  $C > 0$  such that for all  $\eta >$

$0 \{\textstyle \eta > 0\}$

-- 03 --

RMSProp has shown good adaptation of learning rate in different applications. RMSProp can be seen as a generalization of Rprop and is capable to work with mini-batches as well opposed to only full-batches.

-- 04 --

$$\max_{k=0, \dots, \lfloor T/\eta \rfloor} |E[g(w_k)] - E[g(W_{k\eta})]| \leq C\eta^2. \quad \{\displaystyle \max_{k=0, \dots, \lfloor T/\eta \rfloor} |E[g(w_k)] - E[g(W_{k\eta})]| \leq C\eta^2.\}$$

-- 05 --

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update  $\Delta w$  at each iteration, and determines the next update as a linear combination of the gradient and the previous update:

-- 06 --

AdaGrad AdaGrad (for adaptive gradient algorithm) is a modified stochastic gradient descent algorithm with per-parameter learning rate, first published in 2011. Informally, this increases the learning rate for sparser parameters and decreases the learning rate for ones that are less sparse. This strategy often improves convergence performance over standard stochastic gradient descent in settings where data is sparse and sparse parameters are more informative. Examples of such applications include natural language processing and image recognition. It still has a base learning rate  $\eta$ , but this is multiplied with the elements of a vector  $\{G_{j,j}\}$  which is the diagonal of the outer product matrix

-- 07 --

The step size is denoted by  $\eta \{\displaystyle \eta\}$  (sometimes called the learning rate in machine learning) and here " $:= \{\displaystyle :=\}$ " denotes the update of a variable in the algorithm. In many cases, the summand functions have a simple form that enables inexpensive evaluations of the sum-function and the sum gradient. For example, in statistics, one-parameter exponential families allow economical function-evaluations and gradient-evaluations. However, in other cases, evaluating the sum-gradient may require expensive evaluations of the

gradients from all summand functions. When the training set is enormous and no simple formulas exist, evaluating the sums of gradients becomes very expensive, because evaluating the gradient requires evaluating all the summand functions' gradients. To economize on the computational cost at every iteration, stochastic gradient descent samples a subset of summand functions at every step. This is very effective in the case of large-scale machine learning problems.

-- 08 --

**Extensions and variants** Many improvements on the basic stochastic gradient descent algorithm have been proposed and used. In particular, in machine learning, the need to set a learning rate (step size) has been recognized as problematic. Setting this parameter too high can cause the algorithm to diverge; setting it too low makes it slow to converge. A conceptually simple extension of stochastic gradient descent makes the learning rate a decreasing function  $\eta_t$  of the iteration number  $t$ , giving a learning rate schedule, so that the first iterations cause large changes in the parameters, while the later ones do only fine-tuning. Such schedules have been known since the work of MacQueen on k-means clustering. Practical guidance on choosing the step size in several variants of SGD is given by Spall.

-- 09 --

This equation is implicit since  $w^{\text{new}}$  appears on both sides of the equation. It is a stochastic form of the proximal gradient method since the update can also be written as:

-- 10 --

**Linear regression** Suppose we want to fit a straight line  $\hat{y} = w_1 + w_2 x$  to a training set with observations  $((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$  and corresponding estimated responses  $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)$  using least squares. The objective function to be minimized is

-- 11 --

**Second-order methods** A stochastic analogue of the standard (deterministic) Newton–Raphson algorithm (a "second-order" method) provides an asymptotically optimal or near-optimal form of iterative optimization in the setting of stochastic approximation. A method that uses direct measurements of the Hessian matrices of the summands in the empirical risk function was developed by Byrd, Hansen, Nocedal, and Singer. However, directly determining the

required Hessian matrices for optimization may not be possible in practice. Practical and theoretically sound methods for second-order versions of SGD that do not require direct Hessian information are given by Spall and others. (A less efficient method based on finite differences, instead of simultaneous perturbations, is given by Ruppert.) Another approach to the approximation Hessian matrix is replacing it with the Fisher information matrix, which transforms usual gradient to natural. These methods not requiring direct Hessian information are based on either values of the summands in the above empirical risk function or values of the gradients of the summands (i.e., the SGD inputs). In particular, second-order optimality is asymptotically achievable without direct calculation of the Hessian matrices of the summands in the empirical risk function. When the objective is a nonlinear least-squares loss